

## 2. Write a python program to solve 8 Queen's problem

**Aim:** To write a Python program to solve the 8-Queen Problem

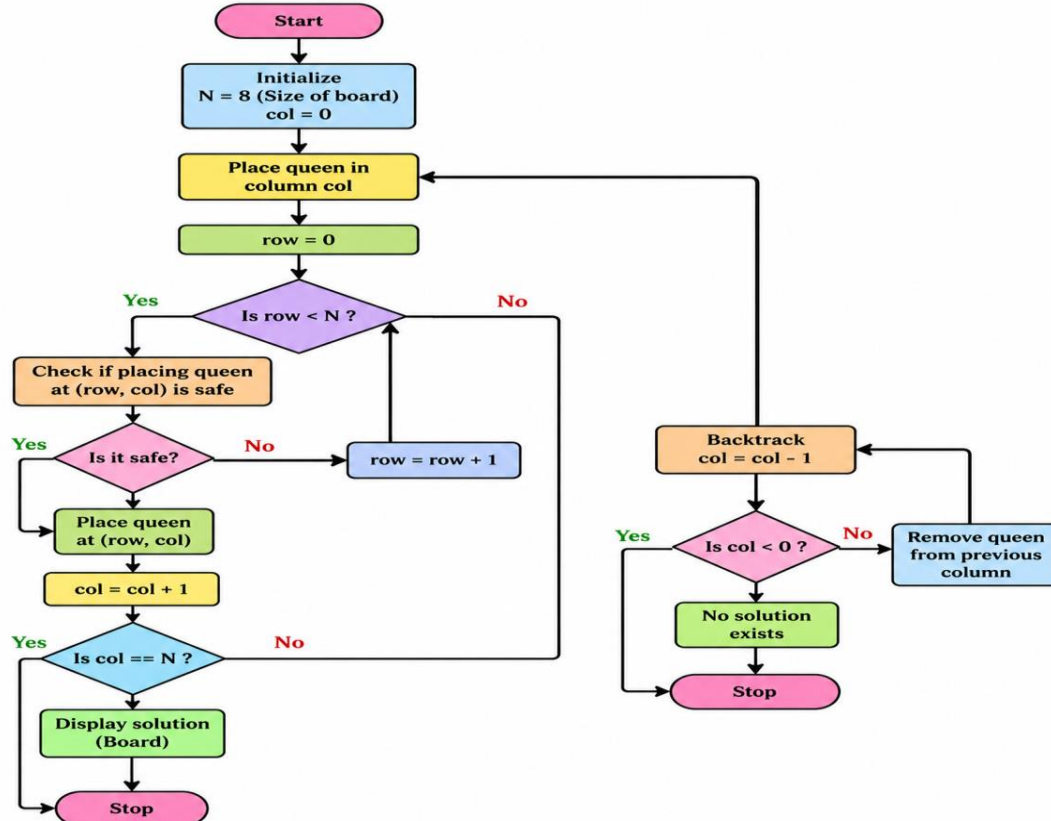
### Objectives:

The objective of this program is to develop a Python program to solve the 8-Queen Problem by placing eight queens on an  $8 \times 8$  chessboard so that no two queens attack each other. The program checks the rows, columns, and diagonals to find a valid arrangement and displays the solution.

### Algorithm:

1. Start.
2. Create an empty  $8 \times 8$  chessboard.
3. Place a queen in the first column.
4. Check whether the position is safe (row and diagonals).
5. If safe, place the queen and move to the next column.
6. If no safe position is found, backtrack and try another row.
7. Repeat until all 8 queens are placed.
8. Display the chessboard with the queen positions.
9. Stop.

### Flow chart:



**Source Code:**

N = 8

```
board = [[0] * N for _ in range(N)]
```

```
def is_safe(row, col):  
    for i in range(col):  
        if board[row][i] == 1:  
            return False
```

```
    i, j = row, col  
    while i >= 0 and j >= 0:  
        if board[i][j] == 1:  
            return False  
        i -= 1  
        j -= 1
```

```
    i, j = row, col  
    while i < N and j >= 0:  
        if board[i][j] == 1:  
            return False  
        i += 1  
        j -= 1
```

```
    return True
```

```
def solve(col):  
    if col >= N:  
        return True
```

```
    for i in range(N):  
        if is_safe(i, col):  
            board[i][col] = 1  
            if solve(col + 1):  
                return True  
            board[i][col] = 0
```

```
    return False
```

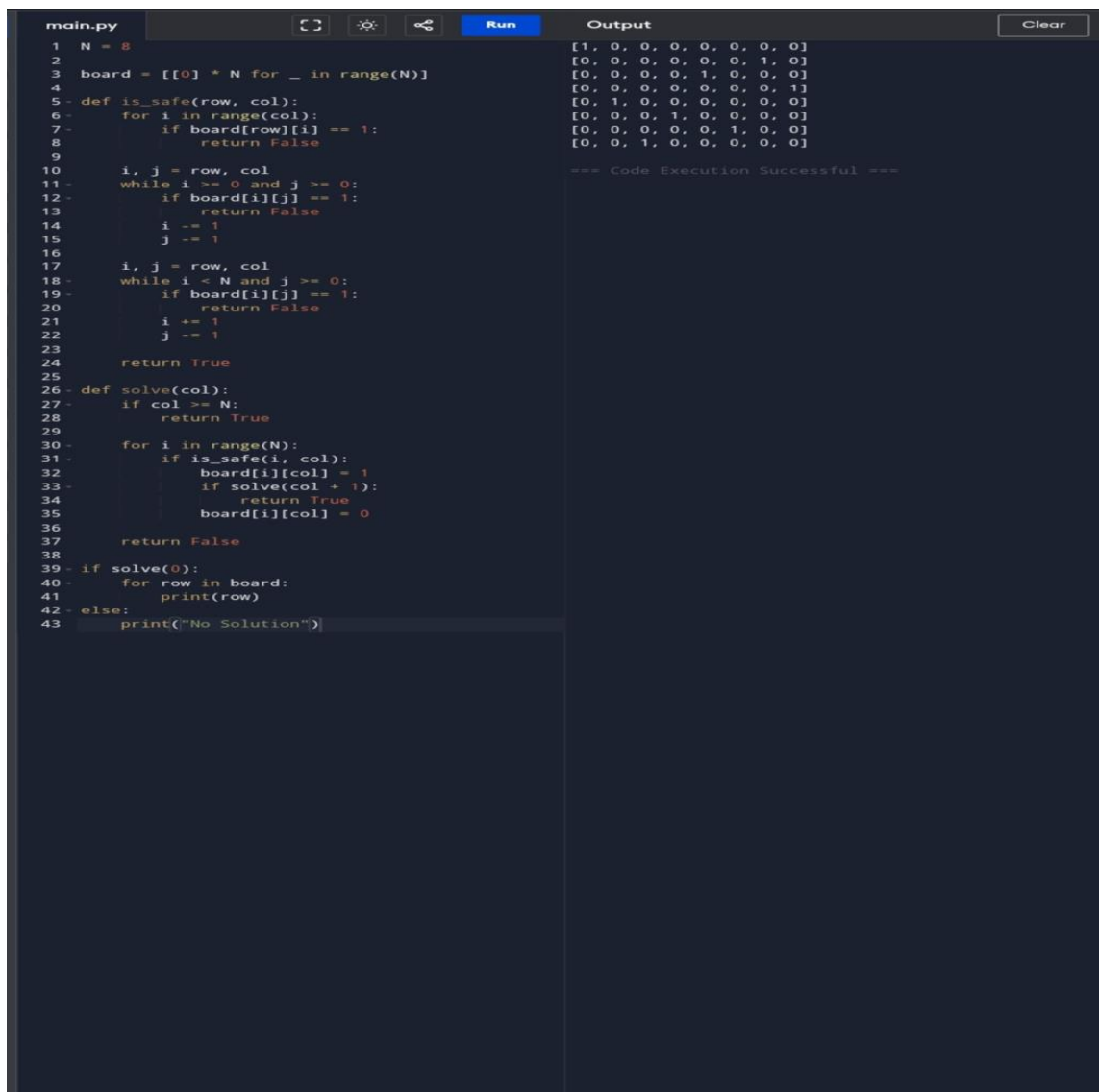
```
if solve(0):  
    for row in board:  
        print(row)  
else:
```

```
print("No Solution")
```

### Result:

The program was executed successfully, and a valid arrangement of 8 queens was obtained on the 8×8 chessboard such that no two queens attack each other.

### Output:



The screenshot shows a code editor with a file named 'main.py' and an 'Output' pane. The code is a Python program to solve the N-Queens problem for N=8. It defines a function 'is\_safe' to check if a queen can be placed at a given position without being attacked by any other queens already on the board. The 'solve' function uses a recursive backtracking approach to find a valid arrangement of queens. The main part of the code calls 'solve(0)' and prints the board if a solution is found, or 'No Solution' if not.

```
1 N = 8
2
3 board = [[0] * N for _ in range(N)]
4
5 def is_safe(row, col):
6     for i in range(col):
7         if board[row][i] == 1:
8             return False
9
10    i, j = row, col
11    while i >= 0 and j >= 0:
12        if board[i][j] == 1:
13            return False
14        i -= 1
15        j -= 1
16
17    i, j = row, col
18    while i < N and j >= 0:
19        if board[i][j] == 1:
20            return False
21        i += 1
22        j -= 1
23
24    return True
25
26 def solve(col):
27     if col >= N:
28         return True
29
30     for i in range(N):
31         if is_safe(i, col):
32             board[i][col] = 1
33             if solve(col + 1):
34                 return True
35             board[i][col] = 0
36
37     return False
38
39 if solve(0):
40     for row in board:
41         print(row)
42 else:
43     print("No Solution")
```

The output pane shows the following 8x8 board configuration, where 1 represents a queen and 0 represents an empty square:

```
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
```

Below the board, the text '=== Code Execution Successful ===' is displayed.

